



A Hands-On

**RAG WITH**

**PYTHON**

Stop Using Frameworks.

Build RAG using **plain Python**.



# What is RAG?

**RAG = Retrieval-Augmented Generation**

**What it does:**

- Finds relevant content from your files
- Feeds that content to the LLM
- Produces grounded answers (less hallucination)

**RAG Pipeline:**

Document → Chunking → Embeddings → Vector DB →  
Retrieval → GPT Answer

## Installation



```
!pip install openai chromadb
```

# Core Components

## File Upload / Load Text:

- Load knowledge from files (TXT, PDF, docs, policies)

## Chunking:

- Break large documents into smaller, meaningful chunks

## Embeddings:

- Convert text chunks into numerical vectors representing meaning

## Vector Database:

- Store embeddings and enable fast similarity search

## Retrieval:

- Find top-K most relevant chunks for a user query

## Generation:

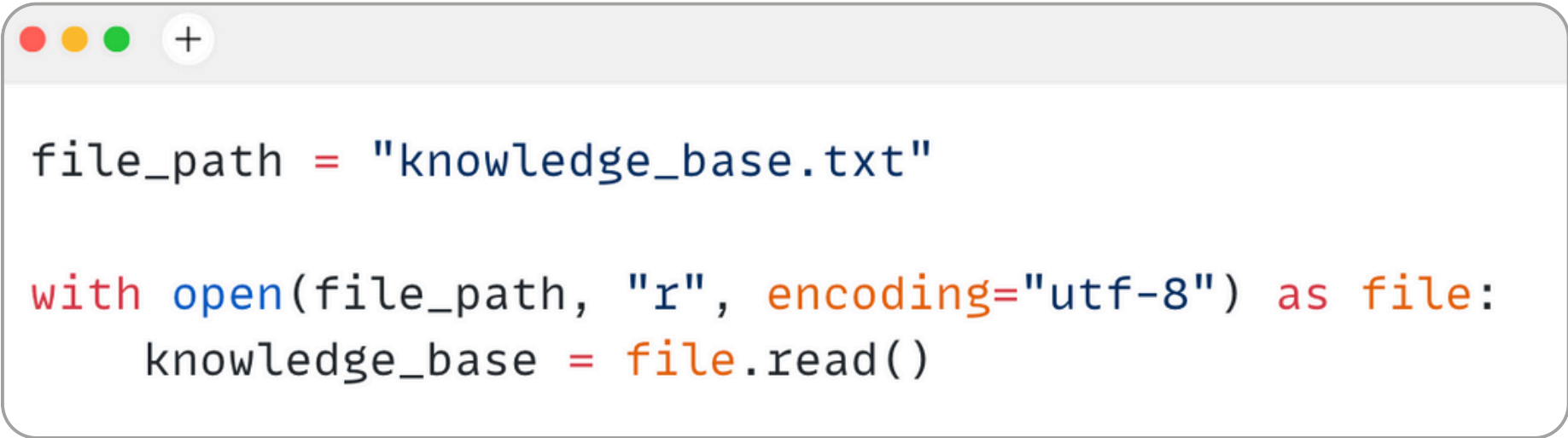
- Generate answers using only the retrieved context

# File Uploading

## What it does:

- Uploads a text file into Colab
- Reads it into a Python string

## Implementation

A code editor window with a light gray header bar containing three colored circles (red, yellow, green) and a plus sign. The main area is white and contains Python code. The code defines a variable 'file\_path' with the value 'knowledge\_base.txt'. It then uses a 'with open' statement to open the file in read mode ('r') with 'encoding="utf-8"', and assigns the result of 'file.read()' to the variable 'knowledge\_base'.

```
file_path = "knowledge_base.txt"

with open(file_path, "r", encoding="utf-8") as file:
    knowledge_base = file.read()
```

# Chunking

## What it does:

- Splits the document into smaller parts (chunks)

## Why needed:

- Embedding models + LLMs have token limits
- Smaller chunks = more accurate retrieval
- Big document → hard to search directly

## Implementation

```
def fixed_word_chunk(text, chunk_size=20):
    words = text.split()
    chunks = []
    for i in range(0, len(words), chunk_size):
        chunks.append(" ".join(words[i:i+chunk_size]))
    return chunks

knowledge_chunks = fixed_word_chunk(knowledge_base, 20)
```

## Embeddings

### What it does:

- Converts each chunk into a vector
- Similar meaning → vectors are closer

### Why needed:

- Vector search finds "meaning", not just keywords

```
from openai import OpenAI
client = OpenAI(api_key="YOUR_KEY")

def get_embeddings(text):
    res = client.embeddings.create(
        input=text,
        model="text-embedding-3-small"
    )
    return res.data[0].embedding
```

# Vector Database (ChromaDB)

## What it does:

- Stores embeddings + original chunks
- Enables fast similarity search

## Why ChromaDB:

- Lightweight + simple
- Runs locally in Colab
- Great for learning + prototypes

## Implementation

```
import chromadb
from chromadb.config import Settings

chroma_client = chromadb.Client(
    Settings(persist_directory="./chroma_store")
)
collection = chroma_client.get_or_create_collection(name="my_kb")
```

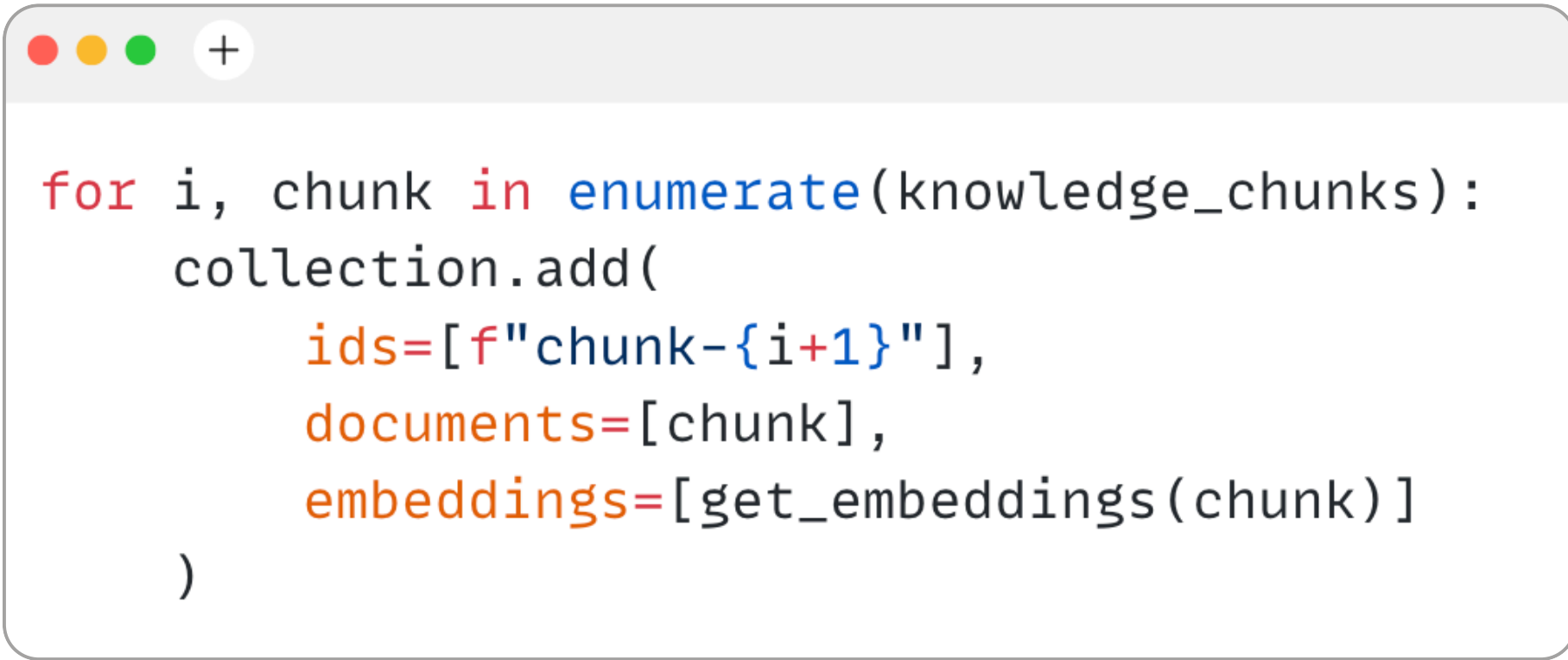
- Client Initialization → Connects Python to ChromaDB
- Persist Directory → Stores vectors locally
- Collection Creation → Holds embeddings for search

# Store Chunks in ChromaDB

## What it does:

- Saves each chunk with:
  - id
  - document
  - embedding

## Implementation



```
for i, chunk in enumerate(knowledge_chunks):  
    collection.add(  
        ids=[f"chunk-{i+1}"],  
        documents=[chunk],  
        embeddings=[get_embeddings(chunk)]  
    )
```

## Behind the scenes:

- Chroma builds a vector index for similarity search

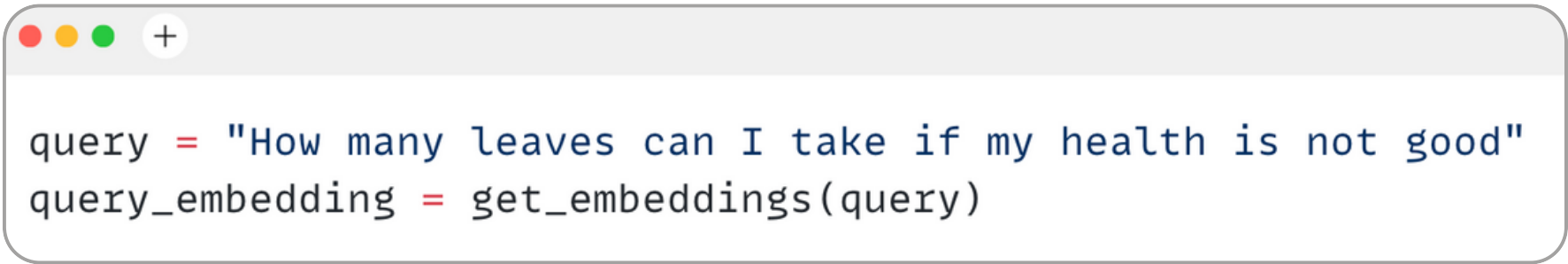


# User Query

## What it does:

- Takes user question
- Converts it to an embedding vector

## Implementation



```
query = "How many leaves can I take if my health is not good"
query_embedding = get_embeddings(query)
```

# Retrieval (Vector Search)

## What it does:

- Finds most relevant chunks using similarity

## How it works:

- Query → embedding
- Compare with stored embeddings
- Return top-k matches



## Implementation

```
results = collection.query(  
    query_embeddings=[query_embedding],  
    n_results=2  
)  
  
top_chunks = results["documents"][0]  
for chunk in top_chunks:  
    print(chunk)
```

## Generation

### What it does:

- GPT answers based ONLY on retrieved context

### System Rule:

- "Use context only. If not present, say you don't know."

## Implementation

```
context = "\n".join(top_chunks)

system_prompt = (
    "Answer based on the given context only. "
    "If the answer isn't in the context, say you don't know."
)

response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user",
         "content": f"Context: {context}\n\nQuestion: {query}"}
    ],
    temperature=0.1,
    max_tokens=300
)
print(response.choices[0].message.content)
```

## Key tips

### Chunking:

- Keep chunks small enough for precision
- Increase size if answers feel incomplete

### Retrieval:

- Try top\_k = 3 to 5 for better coverage

### Generation:

- Keep temperature low for factual answers

# Want more content like this?



Tap that follow button and  
stay in the loop!



Like



Comment



Share



Save